

Universidade da Beira Interior

Departamento de Informática



**Departamento de
Informática**

2º Fase do Projeto Criativo de IHC

Jazzia - Gera, Toca e Jazz!

Elaborado por:

53654 - Bernardete Coelho
54482 - Inês Menezes

Orientador:

Professor Doutor Adriano Raposo

31 de maio de 2026

Resumo

Este relatório técnico apresenta o desenvolvimento da aplicação *Jazzia – Gera, Toca e Jazz!*, criada no âmbito da segunda fase do Projeto Criativo da unidade curricular de Interação Humana com o Computador (IHC). O trabalho surge da necessidade da Generative Jazz, uma empresa fictícia dedicada à música generativa simbólica, que desenvolveu um motor capaz de gerar sequências de acordes, mas que carecia de uma interface gráfica adequada para disponibilizar essa tecnologia aos seus utilizadores. Assim, a nossa equipa foi responsável por conceber e implementar a interface que acompanha este motor.

Após uma primeira fase dedicada ao desenho da interface — incluindo *personas*, fluxogramas, *wireframes* e protótipos de alta fidelidade — esta segunda fase centrou-se na implementação funcional da solução proposta e na elaboração do respetivo manual de utilizador. O objetivo principal consistiu em transformar o protótipo aprovado numa aplicação real, usável e coerente com as boas práticas de IHC.

A aplicação final cumpre todos os requisitos essenciais definidos no enunciado, permitindo ao utilizador obter listas de parâmetros musicais, gerar sequências de acordes, convertê-las em áudio, guardar resultados e gerir as suas sequências. Apesar de limitações de tempo, a interface resultante é intuitiva, estável e próxima do protótipo inicial. Foram ainda aplicadas medidas para melhorar a experiência do utilizador, nomeadamente validações de dados e mecanismos simples de comunicação de erros.

Para além da funcionalidade, procurou-se criar uma experiência estética alinhada com o universo musical, transmitindo ao utilizador a sensação de estar num ambiente criativo: a página inicial remete para um palco, as páginas do estúdio evocam um caderno de composições e a área de conta sugere um espaço de bastidores.

O relatório identifica também várias oportunidades de evolução futura, como melhorias estéticas, maior robustez no tratamento de erros, suporte *offline*, personalização da interface e a possibilidade de adicionar notas às sequências guardadas. Ainda assim, considera-se que a versão atual cumpre plenamente os objetivos definidos e serve adequadamente o público-alvo imaginado.

Conteúdo

Conteúdo	iii
Lista de Figuras	v
Lista de Tabelas	vii
1 Introdução	1
1.1 Enquadramento	1
1.2 Objetivos	2
1.3 Divisão de Tarefas	2
1.4 Organização do Documento	3
2 Tecnologias e Ferramentas Utilizadas	5
2.1 Introdução	5
2.2 Tecnologias Usadas	5
2.2.1 Figma	5
2.2.2 VS Code	6
2.2.3 React.js	6
2.2.4 Node.js	6
2.2.5 GenJazz API	7
2.2.6 Clerk	7
2.2.7 Overleaf	7
2.2.8 GitHub	8
2.3 Conclusões	8
3 Implementação e Testes	9
3.1 Introdução	9
3.2 Implementação	9
3.2.1 Passos Iniciais	9
3.2.2 Tarefas de Bernardete Coelho	11
3.2.2.1 Testes e Depuração	11
3.2.3 Tarefas de Inês Menezes	16
3.3 Diferenças Entre o Protótipo e a Aplicação Final	17

3.4	Elaboração do Manual de Utilizador	19
3.5	Conclusões	19
4	Conclusões e Trabalho Futuro	21
4.1	Conclusões Principais	21
4.2	Trabalho Futuro	22
AI Disclaimer		25
4.3	Observações da Bernardete Coelho	25
Manual de Utilizador		27
4.4	Introdução	27
4.5	Primeiros Passos	28
4.6	Interface da Jazzia	28
4.7	Gerar Acordes e Sequências	28
4.8	Guardar e Gerir Sequências	29
4.9	Gestão de Conta (Clerk)	30
4.10	Perguntas Frequentes (FAQ)	30
4.11	Resolução de Problemas	31
4.11.1	Passos Gerais	31
4.11.2	Erros Comuns da Aplicação	31
4.11.3	Erros Comuns do <i>Clerk</i>	32
4.11.4	Como Contactar o Suporte	32
4.12	Termos, Segurança e Privacidade	32
4.13	Contacto e Suporte	33
4.14	Termos e Condições	33
Bibliografia		35

Lista de Figuras

3.1	Keys que a GenJazz API aceita.	12
3.2	Estruturas que a GenJazz API aceita.	12
3.3	Modulações que a GenJazz API aceita.	13
3.4	Uso de IndexedDB em Jazzia.	14
3.5	Áudios vazios nas progressões geradas pela GenJazz API.	15
3.6	Imagem da página de entrada: copiada (direita) e exportada (esquerda)	16

Lista de Tabelas

1.1	Tabela de Divisão Inicial de Tarefas.	3
3.1	Tabela de Divisão Final de Tarefas.	19

Acrónimos

UBI	Universidade da Beira Interior
IHC	Interação Humana com o Computador
SDK	Software Development Kit

Capítulo

1

Introdução

1.1 Enquadramento

O presente documento é um relatório técnico que descreve a implementação de uma aplicação baseada em React.js, intitulada "Jazzia - Gera, Toca e Jazz!". Esta interface insere-se no âmbito da segunda fase do projeto criativo da unidade curricular Interação Humana com o Computador (IHC), do 2º ano do curso de Engenharia Informática, da Universidade da Beira Interior (UBI).

Para este trabalho, foi-nos dado o seguinte contexto. A Generative Jazz, uma empresa (fictícia) que se dedica à investigação e desenvolvimento na área da música generativa simbólica, concebeu um novo motor generativo que gera harmonias originais (sequências de acordes), para serem usados como base para improvisação por músicos, professores e estudantes de jazz. No entanto, a empresa não dispõe de competências na área do desenho e desenvolvimento de interfaces gráficas e querendo assegurar o sucesso do seu produto, necessita de um parceiro estratégico para desenhar e implementar a interface perfeita (frontend) para o seu novo motor (backend).

Foi neste contexto que a nossa equipa foi chamada a intervir. A nossa tarefa na primeira fase analisar os requisitos de usabilidade e, em conformidade, desenhar uma proposta para a nova interface de aplicação móvel. Para isso, elaborámos cartões de *personas*, fluxogramas, *wireframes*, componentes e protótipos de alta fidelidade funcionais. Nesta segunda fase, uma vez que a Generative Jazz já aprovou a nossa proposta, o foco está na implementação do código da interface que desenhámos e no desenvolvimento de um manual de utilizador.

Encontrar uma solução ótima para o problema da Generative Jazz é importante não só para consolidar os conteúdos da unidade curricular, mas também para desenvolver competências práticas de concepção e implementação de uma interface real. Este projeto permite-nos perceber como um bom design é crucial para tornar acessíveis ao público as tecnologias avançadas, como motores generativos. Num ecossistema digital cada vez mais inteligente e repleto de soluções semelhantes, uma interface bem desenhada torna-se decisiva para que uma aplicação se destaque pela qualidade da experiência que oferece.

1.2 Objetivos

Posto que a primeira fase deste projeto se centrou em todo o processo de desenho de interface já mencionado na secção anterior, esta segunda parte pretende que os seguintes objetivos sejam cumpridos:

- Implementar em React.js a interface da primeira fase, conforme ao protótipo que propusemos;
- Configurar corretamente a autenticação de utilizador com Clerk, apenas com *Sign In*, usando as nossas contas da UBI;
- Aplicar de forma eficaz e eficiente as funcionalidades do motor generativo fornecidas pela Generative Jazz, com os devidos parâmetros, *end-points*, pedidos e respostas.
- Aplicar adequadamente todas as boas práticas de desenvolvimento de interfaces abordadas nas aulas teóricas de IHC.
- Testar e depurar a aplicação, até que esteja usável.
- Elaborar documentação, isto é, redigir um manual de utilizador durante a implementação.

1.3 Divisão de Tarefas

Para tornarmos o desenvolvimento do projeto independente e organizado, fizemos a seguinte divisão de tarefas, semelhante à da primeira fase, que consta na tabela 1.1.

Aluna	Tarefas
53654, Bernardete Coelho	Implementar alguns os <i>pop-ups</i> e as páginas seguintes: "Secção de Conta da Dashboard", "Ver Dados de Conta", "Alterar Dados de Conta", "Mudar de Conta", "Gerar Sequência", e "Ver Sequência Gerada". Escrita da maioria do relatório e das partes relativas às suas páginas no Manual de Utilizador.
54482, Inês Menezes	Implementar alguns os <i>pop-ups</i> e as páginas seguintes: "Entrada", "Registo", "Login", "Secção Estúdio da Dashboard" e "Lista de Sequências Guardadas". Tratar da autenticação usando Clerk. Escrita das suas secções do relatório e das partes relativas às suas páginas no Manual de Utilizador.

Tabela 1.1: Tabela de Divisão Inicial de Tarefas.

1.4 Organização do Documento

De modo a refletir o trabalho que foi feito, este documento encontra-se estruturado da seguinte forma:

1. O primeiro capítulo – **Introdução** – apresenta o projeto, a motivação para a sua escolha, o enquadramento para o mesmo, os seus objetivos e a respetiva organização do documento.
2. O segundo capítulo – **Tecnologias e Ferramentas Utilizadas** – descreve as tecnologias utilizadas durante do desenvolvimento da aplicação, assim como o papel específico de cada uma neste projeto.
3. O terceiro capítulo – **Implementação e Testes** – aborda o processo de implementação da aplicação, por cada membro da equipa. Contém as principais decisões tomadas e também os testes e depuração realizados.
4. O quarto capítulo – **Conclusões e Trabalho Futuro** – remata o conteúdo principal do relatório, com uma conclusão sobre a completude dos objetivos e uma reflexão sobre pontos de melhoria futura.
5. O capítulo **AI Disclaimer** está dedicado à explicitação de quais inteligências artificiais foram usadas neste trabalho e que papel desempenharam.
6. O capítulo **Manual de Utilizador** é um anexo que contém, tal como o seu título sugere, a documentação da aplicação dedicada ao utilizador.

Capítulo

2

Tecnologias e Ferramentas Utilizadas

2.1 Introdução

A escolha das tecnologias de suporte é um passo determinante para conseguir uma implementação ótima e a concretização dos objetivos estipulados. No caso deste projeto, a maioria das tecnologias e *softwares* utilizados não foi escolha da nossa equipa, mas do professor regente da unidade curricular IHC. Respeitámos essas escolhas e procurámos tirar o melhor proveito das ferramentas eleitas.

Este capítulo inclui a descrição de todas as tecnologias de que dispusemos para elaborar este projeto e utilidade de cada uma, aplicada a cada tarefa que este trabalho envolve. A secção 2.2 inclui essas descrições relativas ao Figma, VS Code, React.js, Node.js, GenJazz API, Clerk, Overleaf e GitHub.

2.2 Tecnologias Usadas

2.2.1 Figma

O Figma¹ é uma plataforma de edição gráfica e design de interfaces baseada na Web e colaborativa. Com este software, é possível fazer um pouco de tudo, especialmente a nível gráfico, desde a conceção de sites e interfaces gráficas para aplicações até à criação de publicações nas redes sociais e apresentações. É, portanto, uma ferramenta cada vez mais popular para empresas e estúdios de design gráfico.

Neste projeto, usámos o Figma para criar cartões de *personas*, fluxogramas, *wireframes* e protótipos das páginas da aplicação. Os componentes que criámos foram reutilizados para a implementação da aplicação final.

2.2.2 VS Code

O Visual Studio Code² é um editor de código gratuito, poderoso e multi-plataforma desenvolvido pela Microsoft. Ele oferece preenchimento automático inteligente de código, depuração integrada e controlo de versões com Git. Destaca-se pela personalização através de extensões, uma comunidade muito ativa e suporte para várias linguagens.

Usámos este software como o ambiente de desenvolvimento de integrado principal para escrever, compilar, executar e depurar os nossos ficheiros de código. A sua utilização foi inestimável para a organização adequada e facilitada das pastas e ficheiros e para fazer gestão das versões dos programas.

2.2.3 React.js

O React.js³ é uma biblioteca JavaScript baseada em componentes, amplamente utilizada para criar interfaces de utilizador dinâmicas e interativas. Ele permite desenvolver aplicações web rápidas e escaláveis graças ao uso do Virtual DOM, que otimiza o processo de renderização ao atualizar apenas os elementos necessários. A arquitetura baseada em componentes promove a reutilização e organização do código, enquanto o uso de JSX facilita a escrita de estruturas de interface dentro do próprio JavaScript⁴. Para este projeto, esta foi a biblioteca que usámos para escrever o código fonte de Jazzia.

2.2.4 Node.js

Node.js⁵ é um ambiente de execução JavaScript, amplamente utilizado para desenvolver aplicações rápidas e escaláveis. O Node.js atua num único processo, recorrendo a operações de I/O assíncronas que evitam bloqueios e permitem lidar com milhares de conexões simultâneas. Esta arquitetura torna-o especialmente eficiente para servidores web e aplicações em tempo real. Além disso permite reutilizar a mesma linguagem no desenvolvimento de frontend como no backend.

Esta infraestrutura é precisamente o que permite que bibliotecas como o React.js funcionem de forma eficiente durante o processo de desenvolvimento

de interfaces, não obstante o Node.js não fazer parte da aplicação final entregue ao utilizador. Ele apenas executa o servidor de desenvolvimento, faz compilação e gere dependências.

2.2.5 GenJazz API

A GenJazz API⁶ é um serviço web (fictício) fornecido pela Generative Jazz concebido para gerar automaticamente progressões harmónicas de jazz e convertê-las em ficheiros áudio no formato MP3. Todas as respostas são devolvidas em JSON, o que facilita a integração com aplicações web modernas e a sua conversão para outros tipos de formatos. A API abstrai a complexidade da criação musical algorítmica, disponibilizando um conjunto simples de endpoints que produzem tanto a estrutura harmónica como o respetivo áudio sintetizado. Funciona, assim, como o núcleo funcional da aplicação desenvolvida no âmbito deste projeto.

2.2.6 Clerk

O Clerk⁷ é uma plataforma completa de autenticação e gestão de utilizadores, concebida para simplificar a integração de funcionalidades como login seguro, perfis de utilizador e autenticação multifator. A ferramenta oferece componentes pré-construídos que permitem adicionar rapidamente interfaces de autenticação com elevada qualidade visual, além de suportar frameworks modernas como React.js, para uma integração fluida no processo de desenvolvimento.

O Clerk acompanha as melhores práticas de segurança e disponibiliza Software Development Kit (SDK)s otimizados para melhorar a experiência do programador, sendo portanto uma solução robusta para aplicações que exigem autenticação fiável e gestão eficiente de utilizadores. Essas são precisamente as necessidades da nossa interface.

2.2.7 Overleaf

O Overleaf⁸ é uma plataforma online projetada para ajudar utilizadores na criação, edição e colaboração de documentos científicos, especialmente aqueles que são escritos em LaTeX. Tem uma vasta utilização na produção de artigos académicos, relatórios técnicos, teses, dissertações e outros documentos que exigem formatação precisa e profissional.

Este software foi indispensável para a criação deste relatório técnico. Permitiu-nos acompanhar em tempo real o progresso dos capítulos incumbidos a cada membro e escrever através de qualquer dispositivo, em qualquer lugar com rede.

2.2.8 GitHub

O GitHub⁹ é uma plataforma de desenvolvimento colaborativo que aloja projetos na nuvem, utilizando o sistema de controlo de versões. A plataforma ajuda os criadores a armazenar e administrar código, fazer o registos de mudanças e acompanhar o progresso de ficheiros.

Utilizamos o GitHub para manter o histórico das versões do nosso trabalho, tanto a nível dos ficheiros de código como também deste relatório técnico. Além disso, foi útil também para acompanhar a evolução dos ficheiros encargos a cada membro e transferi-los mais facilmente, para fins de depuração.

2.3 Conclusões

Em suma, as tecnologias utilizadas ao longo deste projeto desempenharam papéis complementares e essenciais. O Figma apoiou a fase de prototipagem e o VS Code serviu como ambiente de desenvolvimento. O React.js permitiu implementar a interface e o Node.js forneceu a infraestrutura necessária para executar o ambiente de desenvolvimento. A GenJazz API foi responsável pela geração das progressões harmónicas e ficheiros áudio e o Clerk assegurou a autenticação e gestão de utilizadores. Por fim, o Overleaf facilitou a escrita deste relatório e o GitHub garantiu o controlo de versões do trabalho.

Em conjunto, estas ferramentas utilizadas ao longo deste projeto desempenharam papéis complementares e essenciais. Sobretudo, permitiram estruturar um processo de desenvolvimento coerente, eficiente e alinhado com os objetivos do projeto, como abordaremos no capítulo seguinte.

Capítulo

3

Implementação e Testes

3.1 Introdução

Tendo já os objetivos definidos e as tecnologias escolhidas, chegou a altura de passar ao planeamento, à implementação e, por fim, à depuração com testes. Neste capítulo detalhamos os primeiros passos que definiram a transição entre a fase um e a fase dois deste projeto criativos, a organização do ficheiro por pastas e as decisões de implementação que tomámos, tanto gerais como mais específicas. Ainda relata testes de depuração realizámos, problemas identificados e o sucesso ou não na conceção de soluções para os resolver.

3.2 Implementação

Para maior clareza e organização, decidimos dividir a esta secção em três subsecções, para tornar a descrição da implementação mais detalhada e precisa. Primeiramente, descrevemos os passos iniciais que tomámos em conjunto, antes de prosseguir com as secções referentes às tarefas individuais.

3.2.1 Passos Iniciais

Breve Planeamento Na fase de planeamento, começámos por distribuir as doze páginas do nosso protótipo entre nós as duas, conforme a divisão de tarefas da tabela 1.1. Criámos o repositório no GitHub para podermos colocar os primeiros ficheiros e desde cedo trabalhar em paralelo, acompanhar o progresso dos ficheiros e gerir e recuperar versões dos ficheiros. Decidimos a organização do projeto por pastas e adicionámos apenas as essenciais ao re-

positório, uma vez que GitHub não lida bem com pastas que incluem mais de cem ficheiros, como é o caso dos módulos criados na inicialização de Node.js.

Organização do Projeto Foi-nos fornecida uma pasta da GenJazz API (genjazzui), com uma estrutura típica de projetos em React.js. Optámos por manter todos esses ficheiros nessa pasta, criámos o projeto jazzia à parte e fomos integrando o conteúdo da genjazzui à medida que precisámos dele nos nossos ficheiros. A organização do nosso projeto sofreu algumas alterações ao longo do tempo, mas no final fizémos o possível para manter todos os ficheiros que implementámos dentro de uma única pasta src. Dentro dela, organizámos os ficheiros em pastas adequadas, para evitar desordenação e dificuldade a encontrar documentos.

Extração de Componentes do Figma O primeiro passo na implementação foi exportar os componentes que fizemos no Figma para uma subpasta da pasta principal dos ficheiros de código e adiciona-la ao GitHub. Criámos alguns ficheiros para estilos e componentes, mediante a necessidade. Com esses ficheiros no Github e com a sua consulta, víamos logo o que já fora implementado e estava pronto a ser utilizado e reutilizado. Isso evitou criarmos os mesmos componentes, por não saber que já existiam. Esses ficheiros, juntamente com o ficheiro de rotas, foram os primeiros a ser elaborados e completados, pois serviram de biblioteca completa de materiais da interface para os restantes ficheiros. Em seguida, começámos a implementar as páginas.

A ordem de trabalho para a primeira fase de implementação de código foi a seguinte. Primeiro, verificar que componentes eram comuns à maioria das páginas e definir o seu estilo e o documento único de formato .jsx para armazenar as componentes globais. Em seguida, analisámos página a página, das nossas tarefas individuais, e fomos retirando do Figma os estilos dos componentes mais específicos de cada página e criando os seus estilos e componentes em React.js.

Começo da Implementação Individual Para criarmos os primeiros componentes e para orientar a escrita do código, consultámos recursos de aprendizagem de React.js, nomeadamente a documentação oficial¹⁰³. Tendo já mencionado as decisões iniciais que tomámos para a primeira fase de implementação, seguem-se as secções que tratam em mais detalhe do procedimento escolhido por cada membro do grupo, visando completar as suas tarefas com eficiência.

3.2.2 Tarefas de Bernardete Coelho

Decisões de Implementação

Nesta parte, vou descrever de forma sucinta a minha forma de trabalhar durante a implementação da interface em React.js. Primeiramente, comecei por identificar que componentes deviam ser exportados e quais poderiam ser feitos diretamente em CSS. Fiz exportação apenas dos componentes mais complexos, nomeadamente os botões da Página de Estúdio, da Página de Lista de Sequências, da barra inferior da *dashboard* e de dois fundos da aplicação (entrada e estúdio), para não tornar a interface pesada. Em seguida, dediquei-me a criar componentes em ficheiro `.jsx`, agrupando-os por páginas.

Primeiro fiz os restantes componentes globais manualmente e, em seguida, à medida que ia tratando de cada página de que estava incumbida, fui implementando os outros componentes, como modais, caixas e tabelas. Tratei do ficheiro *Router.js* logo no começo, bem como da *App.js*. A aplicação correta dos estilos foi morosa no início, especialmente para a página de geração de sequências. Quando terminei a implementação estética das minhas páginas, passei à análise dos endpoints da GenJazz API e à sua integração nas minhas páginas.

Não pude continuar sem ter os ficheiros da Inês prontos, nomeadamente Listar Sequências e a Página de Estúdio. A Inês enviou-me tudo o que fez e eu fiz a integração das nossas implementações. Tive de mudar estilos e ajustar componentes, pois seguimos abordagens ligeiramente diferentes na escrita de código. Terminei as suas páginas e integrei-as com as minhas e só então pude começar a fazer testes e depuração das funcionalidades ligadas ao servidor.

3.2.2.1 Testes e Depuração

Alterações no Círculo de Quintas Durante a elaboração das funcionalidades das páginas "Gerar Sequência" e "Sequência Gerada", aproveitando os endpoints cedidos pela GenJazz API, reparei que a geração corri bem com tonalidades sem caracteres especiais, como o cardinal. Usei a consola do navegador para perceber que tipo de tonalidades, estruturas e modulações são aceites pela API. Analisei os *logs* a nível de formato, onde constatei que ela trata *arrays* de objetos, a nível de valores aceites, onde concluí que a API só aceita 25 tonalidades, e a nível de linguagem, onde confirmei que a API só processa va-

lores em inglês. Deveras, as tonalidades com sustenidos do circulo de quintas não são aceites pela API, segundo a lista que obtive na consola do Chrome.

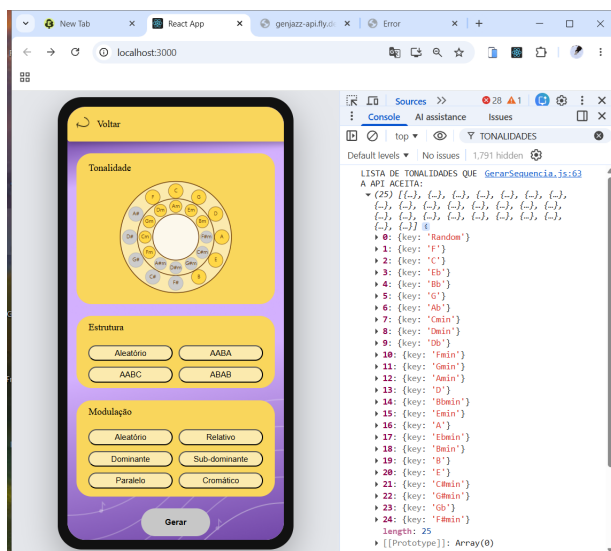


Figura 3.1: Keys que a GenJazz API aceita.

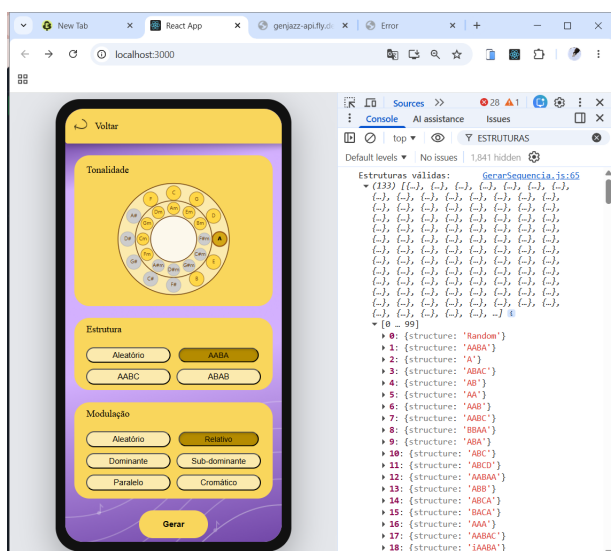


Figura 3.2: Estruturas que a GenJazz API aceita.

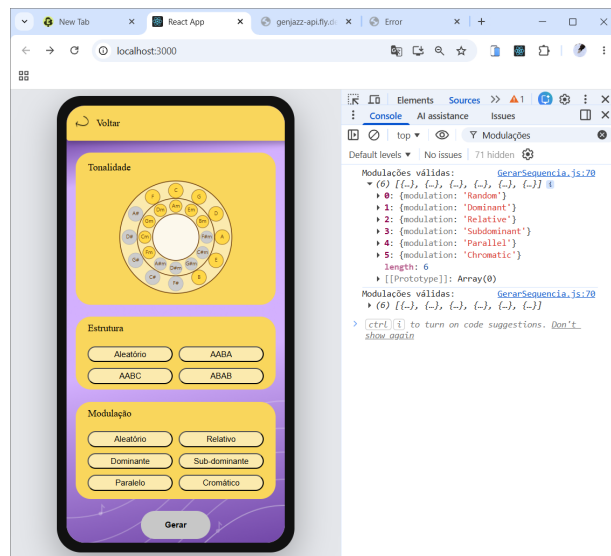


Figura 3.3: Modulações que a GenJazz API aceita.

Para melhorar a experiência do utilizador, desativei no Circulo de Quintas todas as tonalidades proibidas. Ademais, modifiquei os modais que inicialmente tinha definido, para que informassem o utilizador da causa exata do erro, nomeadamente se o problema é de rede, ou de falha na geração. Apesar dessas medidas, ainda há casos em que a progressão falha, segundo os meus testes. A hipótese é que a combinação específica das três escolhas (tonalidade, estrutura e modulação) não existe ou não consegue ser processada pela GenJazz API.

Ao voltar a analisar o código, repensei e ponderei que seria mais acertado usar menus de seleção ao invés de botões, uma vez que a interface têm cento-e-trinta-e-três estruturas. Seria bastante limitante ter apenas quatro botões. Dessa forma, fiz uma uso mais eficiente do código que nos foi fornecido da GenJazz API. Também modifiquei novamente o círculo de quintas e incluí apenas as cifras que o sistema aceita. Não adicionei o botão "Random", porque o utilizador pode simplesmente escolher um uma cifra aleatório. Com estas alterações, circulo apresentado deixou de ser um circulo de quintas. É apenas um círculo de notas.

Persistência das Sequências Guardadas Enquanto estava a implementar o fluxo de guardar sequência e ir vê-la na lista, deparei-me com vários desafios. Primeiramente, quando tinha já feito a parte estética e as funcionalidades básicas, testei guardar uma sequência e recebia sempre o modal de sucesso. No

entanto, mesmo com a página da lista de sequências implementada e supostamente funcional, as sequências não estavam a aparecer. Fiz alguns testes. Procurei saber, através de *logs* na consola do Chrome, se a API estava a devolver algum id. De facto estava, mas a lógica implementada até então não estava a permitir a listagem correta.

Para facilitar a depuração, decidi usar indexedDB no Chrome. Usei mais *logs* na consola de para ver que erros estava a obter, que ids eram passados entre páginas. Os dados persistiram corretamente no armazenamento local e estavam a aparecer na lista de sequências. No entanto, deparei-me com o desafio das designações de cada sequência. A GenJazz API guarda o id, os acordes, a tonalidade, a estrutura e a modulação, mas não guarda o nome de sequências. Pensei então em usar nomes mais genéricos, aproveitando-me do id. Defini "Progressão id - Acordes" para todas, com a atribuição dinâmica desses valores.

Eventualmente, consegui que a API devolvesse as sequências que eu pedira para guardar. Durante esses testes, reparei que para a IndexedDB e a API estarem em conformidade de ids, a primeira tinha de receber `_id` e a segunda id. Mantive a lógica de IndexedDB no código, pois é a única forma de guardar os acordes das sequências juntamente com o nomes definidos pelo utilizador. Considero já a possibilidade de um dia a GenJazz API ser expandida e Jazzia poder ser melhorada, com base nessas mudanças.

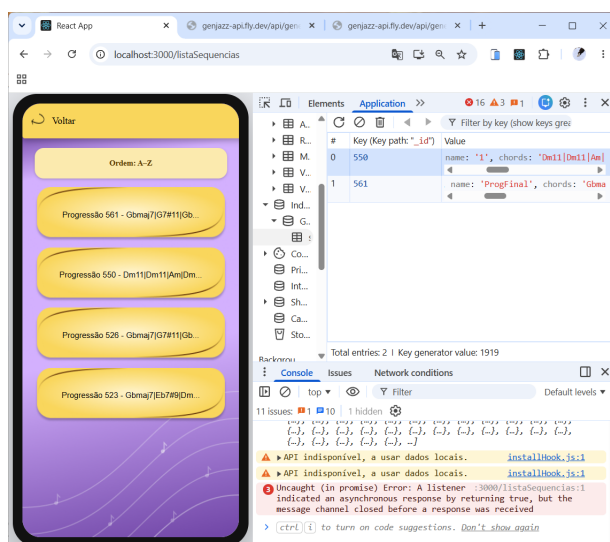


Figura 3.4: Uso de IndexedDB em Jazzia.

Ainda neste processo, deparei-me com o carregamento corretos da lista de sequências guardadas, mas com tabela de cifras vazia quando eu clicava numa dela. Novamente, precisei de analisar as mensagens da consola para perceber se o problema era o formato de dados que a IndexedDB armazena ou erros na ligação à API. Reestruturei novamente o código e deixei-o preparado para a possibilidade de receber valores nulos, evitando que a tabela desapareça e fique apenas a caixa que a contém. No entanto, esse é um tipo de erro que pode acontecer e que tive em conta no Manual de Utilizador.

Áudios que não reproduzem Já com a página de gerar sequências bem implementada, do ponto de vista dos valores aceites pela GenJazz API, experimentei gerar várias. No processo, acabei por usar `ClipLoader` do React para tornar a espera mais interessante. Reparei que para algumas progressões geradas, o controlador de áudio aparecia, mas o tempo estava a zero. Isso não acontece em todas, pelo que acredito que seja mais uma limitação do servidor. Provavelmente, há combinações de tonalidades, modulações e estruturas que a API ainda não está pronta para processar. Não me dediquei a tratar esse erro com modais, mas lembrei-me de o adicionar ao Manual de Utilizador.

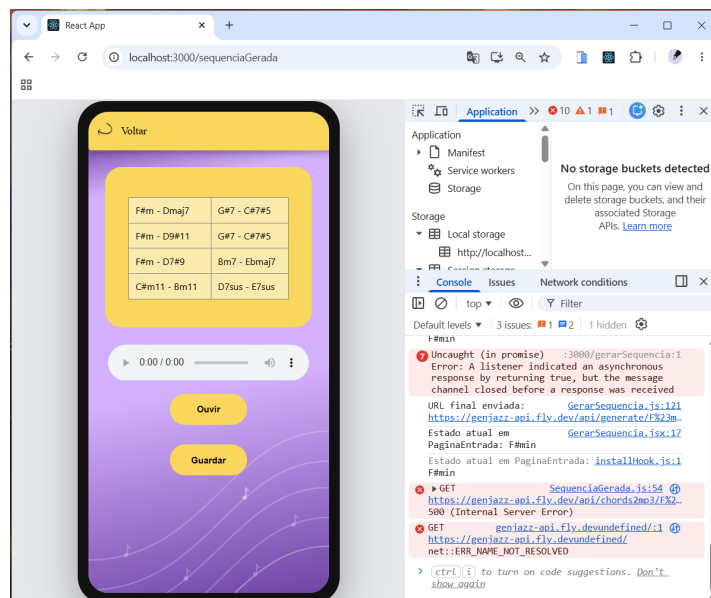


Figura 3.5: Áudios vazios nas progressões geradas pela GenJazz API.

3.2.3 Tarefas de Inês Menezes

Decisões de Implementação

A implementação inicial das páginas de login, registo e alteração de password mantinham a estilização do protótipo Figma, porém foram removidas para manter a utilização da plataforma de autenticação Clerk de acordo com a aplicação base.

Testes e Depuração

Utilização direta de designs do protótipo Figma A criação dos elementos interativos (botões) e importação dos **backgrounds** das páginas mostrou-se enigmática ao início devida à presença de artefactos na imagem, aparentemente, sem nenhuma explicação, mas a pesquisa por recursos sobre como exportar estilos e imagens do Figma revelou que o erro estava na cópia em formato PNG dos elementos e não na sua exportação via a função nativa de exportar da ferramenta Figma.



Figura 3.6: Imagem da página de entrada: copiada (direita) e exportada (esquerda)

3.3 Diferenças Entre o Protótipo e a Aplicação Final

Dedicamos esta secção a todas as mudanças que tivemos de fazer na nossa interface, em relação ao nosso protótipo da Fase 1 deste Projeto Criativo, pois foram bastantes. No início desta Fase 2, acreditávamos que iríamos implementar todas as páginas que tínhamos definido para a nossa interface. No entanto, depois de analisar os *endpoints* fornecidos pela GenJazz API e de perceber as limitações do Clerk a nível de acesso aos dados de utilizador, tomámos a decisão de reduzir a interface apenas às páginas necessárias. Ficámos apenas com as seguintes, as quais por sua vez sofreram também alterações internas nas funcionalidades.

- Página de Entrada – Retirámos todos os botões e usámos uma animação de entrada de três segundos, que leva o utilizador diretamente à página de Login. Retirámos a possibilidade de registo de novos utilizadores.
- Página de Login – Usou os estilos do Clerk ao invés dos nossos.
- Página de Estúdio – Manteve-se igual à do protótipo.
- Página de Conta – Foi simplificada, para mostrar apenas o email e permitir ou utilizador fazer Logout.
- Página de Gerar Sequência – Retirámos os botões de escolha de modulação e estrutura e substituímo-los por menus de seleção. O botão "Gerar" só fica disponível depois da escolha das três opções, algo que não tínhamos previsto na Fase 1.
- Página de Sequência Gerada – Mantivemos o mesmo conteúdo, a disposição dos botões é que mudou. Ao invés de ficarem lado a lado, estão um sobre o outro.
- Página de Sequência Guardada – A estrutura desta é muito semelhante à da página supra referida. No nosso protótipo consta uma caixa para adicionar notas, mas foi retirada da interface final, pois exigia mais trabalho extra de programação e não tivemos tempo para nos dedicarmos a ele.
- Página de Lista de Sequências Guardadas – O conteúdo é o mesmo que o do protótipo, mas não pudemos manter os nomes personalizados pelo utilizador como queríamos.

E as páginas que não implementámos e os motivos para essa decisão seguem-se abaixo.

- Página de Registo – No enunciado do trabalho, estava explícito que não era suposto a interface permitir *Sign Up*. Seguimos essa recomendação.
- Página de Ver Dados de Conta – Devido a tempo limitado, como não pudemos implementar *endpoints* adequados para tratar dados de utilizador e como isso não fazia parte dos requisitos do enunciado, optámos excluir esta página.
- Página de Alterar Dados de Conta – A meio do desenvolvimento, tivemos a ideia de tornar possível ao utilizador alterar os seus dados directamente pela página da sua conta. Portanto, esta página fazia sentido no protótipo, mas não na implementação real.
- Página de Mudar de Conta – No protótipo considerámos que o utilizador poderia ver o histórico de contas no seu dispositivo. No entanto, por questões de privacidade e também porque o Clerk já faz muito bem o trabalho de gestão de contas, excluimos esta página da implementação.

Na secção 1.3 entregámos a tabela de divisão de tarefas inicial. Com o decorrer da implementação e com a simplificação da nossa interface, que sofreu várias perdas de páginas que eram extras que não teríamos tempo de implementar, a divisão de tarefas final acabou com ser a seguinte, que consta na tabela 3.1. Como há muitos ficheiros na nossa interface e para não termos de os mencionar a todos, no topo de cada um consta o número de aluno da sua autora.

Aluna	Tarefas
53654, Bernardete Coelho	Implementação dos ficheiros: CirculoQuintas.jsx, componentesGlobais (css e jsx), Contas.css, GerarSequencia (css e jsx), Modais (css e jsx), SeccaoConta.jsx, SequenciaGerada (css e jsx), Router.js, GerarSequencia.js, SeccaoConta.js, SequenciaGerada.js e SequenciaGuardada.js, SeccaoConta.js. Escrita da maioria do relatório e de algumas das suas partes relativas às suas páginas no Manual de Utilizador.
54482, Inês Menezes	Implementação dos seguintes ficheiros: AltPaginaEntrada.js, Login.js, Tratar da autenticação usando Clerk. Escrita das suas secções do relatório da maioria do Manual de Utilizador.

Tabela 3.1: Tabela de Divisão Final de Tarefas.

3.4 Elaboração do Manual de Utilizador

Para a elaboração do manual de utilizador, começámos por realizar uma pesquisa sobre boas práticas de documentação, com o objetivo de compreender que elementos devem ser incluídos e quais devem ser evitados. Esta revisão inicial baseou-se em diferentes recursos online^{11 12 13}. Durante testes e depuração, deparámo-nos com vários erros que contribuíram para compreender que tipo de problemas os utilizadores poderiam enfrentar e incluímos na documentação. Estruturamo-la de forma semelhante, mas adaptada, à citada neste site do LinkedIn¹⁴ sobre esse assunto.

Só se tornou claro o tipo de problemas que devíamos documentar quando começamos a desenvolver a nossa aplicação. Baseando-nos em todos os testes e depuração e nos desafios que enfrentámos, elaborámos o breve manual de utilizador, que está incluído em anexo, no capítulo 4.3. De modo geral, Jazzia não é uma interface complexa, mas ainda assim tem várias suscetibilidades a erros. Fizemos o possível para mantê-la robusta contra eles, mas reconhecemos que há pontos fracos que devem ser tidos em conta.

3.5 Conclusões

Ao longo deste projeto, a implementação prática revelou-se tão desafiante quanto enriquecedora. A transição do protótipo para código real obrigou-nos

a tomar decisões constantes, ajustar estratégias e repensar soluções sempre que surgiam limitações técnicas, quer da própria API, quer das ferramentas que utilizávamos. A depuração contínua, apoiada em testes sistemáticos e na análise de *logs*, permitiu-nos compreender melhor o comportamento da aplicação e identificar pontos frágeis que não eram evidentes numa fase inicial. Este processo iterativo consolidou não só a robustez da aplicação, mas também a nossa capacidade de adaptação e resolução de problemas.

O desenvolvimento de Jazzia foi um exercício completo de planeamento, organização, integração de componentes e tomada de decisões fundamentadas. A experiência acumulada — desde a gestão do repositório, à extração de elementos do Figma, ao tratamento de erros da API e à persistência de dados — permitiu-nos construir uma aplicação funcional, coerente e preparada para evoluir. Apesar das limitações identificadas, sentimos que alcançamos uma solução sólida e bem estruturada, refletindo o esforço conjunto e a aprendizagem contínua que marcaram todas as fases do projeto.

Conclusões e Trabalho Futuro

4.1 Conclusões Principais

Com a finalização da implementação da interface Jazzia, um processo cheio de desafios, podemos afirmar com confiança que conseguimos implementar as seguintes funcionalidades, descritas como obrigatórias no enunciado do trabalho:

- Obter listas de tonalidades, estruturas e tipos de modulação.
- Geração de sequências de acordes.
- Conversão de sequência de acordes textual em ficheiro de áudio (MP3).
- Guardar sequências de acordes.
- Listar sequências de acordes guardadas.
- Apagar sequência de acordes guardada.

Embora tivéssemos pretendido ir um pouco além dos requisitos mínimos, por falta de gestão de tempo e outros trabalhos de outras unidades curriculares, mantivemos a nossa interface simples e apenas com as funcionalidades pedidas. Acreditamos que Jazzia é uma interface fácil de usar e intuitiva, que cumpre bem o seu propósito e que está muito semelhante ao protótipo que elaborámos na Fase 1 deste Projeto Criativo. Segundo os nossos testes finais, é uma aplicação estável e que funciona de forma eficaz e eficiente, quando há rede estável.

Fizemos o possível para nos precavermos de usos abusivos dos *endpoints* dados. Tivemos, ainda assim, de fazer alterações neles para melhorá-los, de

forma tornar a interface mais robusta. Alguns dessas alterações incluíram validações adicionais de dados introduzidos pelo utilizador e também o uso de IndexedDB para uma camada adicional de persistência de dados no Browser, em caso de falha da API. Fizémos o possível para manter o utilizador informado dos erros que podem ocorrer, usando modais para comunicar com ele e deixando *logs* na consola do navegador, para utilizadores mais curiosos.

Jazzia teve como objetivo dar ao utilizador a sensação de que acompanha a sua vida de músico. Como isto, queremos dizer que a página inicial faz lembrar o cenário épico de um palco, como uma entrada em grande, as páginas do estúdio sugerem a ideia de um caderno de composições e a página de conta evoca a noção de estar nos bastidores. Acreditamos que conseguimos concretizar esse objetivo, mas qualquer alteração futura que venha a ser feita em Jazzia visaria sempre aprimorar essa estética e esse acolhimento que os músicos devem sentir quando a usam. Afinal, compor deve ser um prazer e as interfaces devem trazer sentimentos positivos a quem as usa, tal como aprendemos nas aulas desta unidade curricular.

4.2 Trabalho Futuro

Jazzia é atualmente uma aplicação sólida, mas tem naturalmente vários pontos em que poderia ser melhorada. Primeiramente, a nível estético, reparámos em alguns elementos ligeiramente desalinhados na página de geração de sequências, um aspeto que não conseguimos resolver a tempo. Ademais, por vezes há falhas na reprodução de áudio, algo que não acreditamos ser um problema da nossa interface, mas da GenJazz API. Ainda assim, poderia ser melhorado com um modal que informasse ao utilizador a causa da inexistência de som no reproduzidor. Esses são casos particulares, mas, de modo geral, poderíamos tornar a interface mais robusta e comunicativa com uso de mais tratamento de erros. Seria ainda interessante que a aplicação permitisse uso *offline* e estivesse preparada para quedas de rede.

Acreditamos no potencial de expansão da GenJazz API e de Jazzia e, no caso de isso acontecer, gostaríamos de conceder ao utilizador a possibilidade de alterar os seus dados pessoais com mais detalhe e de permitir-lhe adicionar notas às suas sequências guardadas. Esse último aspeto seria especialmente importante, pois a composição de músicas envolve sempre apontamentos e seria assaz útil ter essa funcionalidade integrada diretamente na interface. Gostaríamos ainda de implementar mais personalização na interface, para que as personas que imaginámos como público alvo se pudessem sentir "em

casa” enquanto usam a aplicação. No entanto, na sua versão atual, acreditamos que Jazzia já cumpre os objetivos para todas essas personas e, por esse motivo, estamos satisfeitas com tudo o que conseguimos desenvolver.

AI Disclaimer

4.3 Observações da Bernardete Coelho

Para o desenvolvimento deste trabalho, usei o Gemini como o meu explicador de React.js. Sendo uma linguagem com a qual não estava familiarizada, ainda que tivesse alguma experiência com JavaScript e CSS, recorri a essa AI para perceber que erros eu estava a cometer no código, de que maneiras poderia implementar as ideias que eu tinha, mas que não sabia escrever corretamente. Usei-a para fazer depuração comigo, para perceber causas de erros, para discutir ideias de implementação e para gerar algum código complexo, que eu teria demorado muito tempo a desenvolver.

Com tempo escasso, o Gemini poupou-me horas de tentativa erro. Eu guiei-o, dizendo-lhe sempre exatamente o que queria, pedi explicações e alternativas, expus-lhe o meu raciocínio para perceber se estava a pensar bem e segui bastantes correções e sugestões dele. No entanto, como todas as AIs, nem sempre as suas sugestões estavam corretas para o que eu precisava e nesses casos deixei sempre a minha opinião prevalecer e fiz eu exatamente o que queria. Deixei ao Gemini a parte de implementar algumas partes de código mais complexo, que analisei sempre antes de implementar, mas a sempre que pude, implementei eu o código, para aumentar o meu conhecimento e aptências de programação em React.js.

Manual de Utilizador

4.4 Introdução

O que é a aplicação Jazzia? A aplicação "móvel" Jazzia é uma aplicação de geração de acordes musicais, sendo estes acordes personalizáveis em termos de tonalidade, estrutura e modulação, através da ferramenta de geração de acordes proporcionada pela empresa GenerativeJazz.

Qual o objetivo da aplicação Jazzia Em geral, a aplicação Jazzia tem como objetivo proporcionar a possibilidade de criar, guardar e revisitar acordes musicais, podendo estes serem utilizados como base ou parte integrante de projetos musicais externos. O objetivo estético é fazer o músico sentir-se em casa, com uma experiência completa de palco, caderno de composições e bastidores.

Qual o público-alvo da aplicação? A Jazzia é uma aplicação intuitiva e facilmente utilizável por utilizadores de diversos perfis, no entanto, destina-se principalmente a músicos e estudantes que procuram uma forma de diversificar, aprender ou criar acordes para projetos tão diversos como uma simples sessão de prática autodidata ou uma produção musical profissional em estúdio.

Quais os requisitos para utilizar Jazzia?

- Browser: Google Chrome (versão 148.0.7778.181 ou superior) ou equivalente.
- Plataforma: Windows, MacOS e Linux.
- Acesso à internet.
- Versão 18 ou superior de Node.js instalada.

4.5 Primeiros Passos

- Como criar conta (Clerk). - Como iniciar sessão. - Como recuperar palavra-passe.
- Como navegar pela interface principal (breve visão geral).

4.6 Interface da Jazzia

- **Área de geração de acordes:** A geração de acordes pela aplicação resume-se aos seguintes passos:
 1. Escolha da tonalidade do círculo de quintas.
 2. Escolha da estrutura do acorde no menu.
 3. Escolha da modulação do acorde no menu.

(Tanto a escolha da estrutura como da modulação pode ser feita de entre várias opções, incluindo a opção aleatória)
- **Área de sequências guardadas:** A área de sequências guardadas contém, sob a forma de um menu rolável que exhibe cada acorde musical guardado em botões retangulares, mostrando o número do acorde e uma pré-visualização da sua sequência.
- **Menu de utilizador (perfil, logout):** O menu de utilizador, acessível pela página principal, contém um cartão com o endereço de e-mail do utilizador ligado no momento e um botão para sair da aplicação, que leva o utilizador de volta ao ecrã inicial ao ser clicado.

4.7 Gerar Acordes e Sequências

De seguida, apresentamos uma descrição da principal funcionalidade da aplicação: a criação de acordes musicais.

- **Como gerar uma sequência:**
 1. Escolher a tonalidade (maior ou menor) do acorde no círculo de quintas.
 2. Escolher a estrutura de entre as seguintes opções (Aleatório, A, ABAC, AB, AA, AAB, AABC, BBAA, ABA)
 3. Escolher a modulação do acorde de entre as seguintes opções (Aleatório, Dominante, Sub-Dominante, Paralelo, Relativo e Cromático).

(Clicando no botão "Gerar" um *loading screen* é visível até que a sequência de acordes seja gerada e o utilizador seja redirecionado para um ecrã que mostra as características da mesma.)

- **Como guardar sequencias:** Após gerar uma sequência, o utilizador é redirecionado para um ecrã que mostra a sequência na sua íntegra, assim como, algumas ações possíveis, entre elas a de guardar a sequência, clicando no botão "Guardar" este apresenta, ao utilizador, um pop-up, onde é pedido para definir o nome da sequência, após clicar em "ok", a sequência é guardada e um segundo pop-up confirma o sucesso da operação, caso não tenha sido possível guardar a sequência, tal acontecimento é transmitido ao utilizador também através de um pop-up.
- **Como ver uma sequência:** No ecrã de menu, acessível pela aba estúdio ou, utilizando os botões de retrocesso, encontra-se o botão "Sequências Guardadas", clicando-lhe o utilizador é redirecionado para um menu que contém todas as sequências guardadas até então, clicar em qualquer uma delas apresentará o ecrã de visualização da sequência escolhida.
- **Como limpar/reiniciar a geração:** Volte atrás na página e volte a clicar no botão "Gerar Sequência" no Estúdio.

4.8 Guardar e Gerir Sequências

1. **Como guardar uma sequência:** As sequências podem ser guardadas no fim da sua geração, clicando no botão "Guardar" presente no ecrã de visualização da sequência criada.
2. **Como atribuir nome:** O nome é atribuído quando uma sequência é guardada, inserindo o nome pretendido no campo de texto destinado. As progressões são apresentadas com o valor numérico do seu ID e não pelo nome que lhe atribuiu, por agora. Não pode alterar isto, mas pode consultar a indexedDB do seu browser para mais detalhes.
3. **Como eliminar:** As sequências guardadas podem ser eliminadas a partir do menu de visualização, acessível clicando numa sequência contida no menu de "Sequências Guardadas" aí, clicando na última opção - "Eliminar", é apresentado ao utilizador um modal que pergunta se o mesmo tem a certeza que deseja eliminar a sequência em questão, as duas opções possíveis "Não" e "Sim" encontram-se indicadas a vermelho e verde respetivamente.

4.9 Gestão de Conta (Clerk)

Neste momento, as opções de gestão de conta na aplicação são bastante limitadas — é possível iniciar e terminar sessão com o email institucional da universidade UBI ou um email da Google, assim como visualizar a conta que se encontra ligada a partir da secção "Conta", acessível pela barra inferior na página principal. Isto deve-se ao facto de parte da interface de autenticação ser gerida externamente pelo Clerk, o que condiciona o nível de controlo disponível diretamente na aplicação.

4.10 Perguntas Frequentes (FAQ)

1. "Não consigo iniciar sessão"

A plataforma Clerk que promove as operações de *login* da nossa aplicação pode não ver a sua conta como autorizada, em especial se for uma conta não institucional UBI. Nesse caso, tente outro modo de autenticação, nomeadamente usando uma conta Google.

2. "A sequência não está a tocar"

Verifique a sua ligação à rede. Caso esteja conectado e a sequência não reproduza, a causa provável é que o servidor não consegue processar esse pedido, pois não existe. Isso não é um erro da aplicação nem do servidor, apenas uma limitação do nosso serviço.

3. "Não consigo guardar"

Verifique a sua ligação à rede. Caso esteja conectado e não consiga guardar a sequência, analise a mensagem de erro do modal que lhe aparece. Se for problema do servidor, tente mais tarde. Se o erro não foi especificado, contacte o serviço de apoio de Jazzia.

4. "Perdi os meus dados"

A interface Jazzia está preparada para armazenar dados em IndexedDB e no seu email. Verifique a sua ligação à rede. Caso esteja ligado, reinicie a aplicação ou recarregue a página. Consulte a IndexedDB do seu browser, caso esteja a usar Jazzia nesse modo, e verifique se os seus dados estão lá. Caso o erro persista, o servidor pode estar indisponível e, nesse caso, tente mais tarde ou contacte o serviço de apoio de Jazzia.

5. "Os valores da tabela da sequência guardada não aparecem"

Verifique a sua ligação à rede. Caso esteja conectado e a tabela estiver vazia ou nem aparecer, tente novamente mais tarde. Pode ser um problema de servidor indisponível. Caso suspeite de perda de dados, faça *logout* e *login* e tente novamente. Verifique ainda a IndexedDB do seu browser, caso esteja a usa-la em modo web. Se suspeita de perda de dados, contacte os serviços de apoio de Jazzia.

6. "O que fazer se a página não carregar?"

Caso a página não esteja a carregar, o erro mais provável é de conexão. Verifique a sua conexão à internet e tente de novo. Se ainda assim a página não carregar, reinicie-a e tente novamente. Caso o erro persista, volte mais tarde ou informe o serviço de apoio de Jazzia.

4.11 Resolução de Problemas

4.11.1 Passos Gerais

- **Recarregar a página:** Em caso de comportamento inesperado, recarregue a página antes de tentar qualquer outra solução.
- **Limpar a *cache* do navegador:** Dados em *cache* desatualizados podem causar erros de carregamento ou de sessão. Limpe a *cache* e os *cookies* nas definições do navegador.
- **Verificar a ligação à internet:** Confirme que existe uma ligação estável. A aplicação e o *Clerk* requerem conectividade para funcionar corretamente.

4.11.2 Erros Comuns da Aplicação

- **Página não carrega ou fica em branco:** Recarregue a página ou tente num navegador diferente. Se o problema persistir, verifique se o serviço está operacional.
- **Conteúdo não atualiza após início de sessão:** Termine a sessão, limpe a *cache* e volte a iniciar sessão com o email institucional.

- **Acesso negado a funcionalidades:** Certas funcionalidades podem estar restringidas à sua conta institucional. Confirme que iniciou sessão com o email correto.
- **Erro no servidor:** O servidor pode estar temporariamente indisponível. Tente mais tarde.

4.11.3 Erros Comuns do *Clerk*

- **Sessão expirada:** A sessão termina automaticamente por inatividade. Inicie sessão novamente para continuar.
- **Código de autorização inválido ou expirado:** Ocorre quando o processo de autenticação demora demasiado ou é repetido. Feche o ecrã de autenticação e tente novamente.
- **Email não reconhecido:** Apenas endereços de email institucionais válidos são aceites. Confirme que está a utilizar o email correto da universidade.
- **Serviço do *Clerk* indisponível:** Em caso de falha do serviço externo, a autenticação ficará temporariamente indisponível. O estado do serviço pode ser consultado em <https://status.clerk.com>.

4.11.4 Como Contactar o Suporte

- **Email:** Para questões relacionadas com a aplicação, contacte a equipa de desenvolvimento através do email disponibilizado no repositório do projeto.
- **GitHub Issues:** Erros e sugestões podem ser reportados diretamente na página de *issues* do repositório oficial no *GitHub*.
- **Problemas com o *Clerk*:** Questões relativas à autenticação que sejam da responsabilidade do *Clerk* devem ser reportadas através do suporte oficial em <https://clerk.com/support>.

4.12 Termos, Segurança e Privacidade

- **Aviso sobre utilização de dados pelo *Clerk*:** A autenticação é gerida pelo *Clerk*, um serviço externo que recolhe e processa dados pessoais como endereços de email, informações de sessão e dados de utilização.

A aplicação atua como responsável pelo tratamento dos dados, sendo o *Clerk* o subcontratante. Para mais informações, consulte a política de privacidade do *Clerk* em <https://clerk.com/legal/privacy>.

- **Limitações da aplicação:** A aplicação não armazena palavras-passe nem gere diretamente credenciais de autenticação. As funcionalidades de gestão de conta são limitadas pelo âmbito do *Clerk* e pelas políticas da instituição. A disponibilidade do serviço depende parcialmente de infraestrutura de terceiros.
- **Direitos do utilizador:** Em conformidade com o Regulamento Geral sobre a Proteção de Dados (RGPD), o utilizador tem direito de acesso, retificação e eliminação dos seus dados pessoais. Para exercer estes direitos relativamente aos dados tratados pelo *Clerk*, deverá contactar diretamente o responsável pela aplicação ou o próprio *Clerk* através dos canais oficiais.

4.13 Contacto e Suporte

- **Email:** bernardete.mm.coelho@ubi.pt · ines.s.menezes@ubi.pt
- **GitHub:** <https://github.com/BernaCoe/Projeto-Criativo-IHC-53654-54482>

4.14 Termos e Condições

Este documento está licenciado sob a Creative Commons Atribuição 4.0 Internacional (CC BY 4.0). Para mais informações, consulte o site oficial¹⁵.

Bibliografia

- [1] Pixartprinting. Figma: o que é e como funciona?, 2026. <https://www.pixartprinting.com.pt/blog/figma-que-e/>. Último acesso a 28 de Maio de 2026.
- [2] InformaTec Digital. O que é o Visual Studio Code e para que serve?, 2026. <https://informatecdigital.com/pt/0-que-%C3%A9-o-Visual-Studio-Code/>. Último acesso a 15 de Maio de 2026.
- [3] React Team. Referência da API do React, 2024. [Online] <https://pt-br.react.dev/reference/react>. Último acesso a 26 de Maio de 2026.
- [4] GeeksforGeeks. ReactJS Introduction, 2024. [Online] <https://www.geeksforgeeks.org/reactjs/reactjs-introduction/>. Último acesso a 26 de Maio de 2026.
- [5] OpenJS Foundation. Introduction to Node.js, 2024. [Online] <https://nodejs.org/learn/getting-started/introduction-to-nodejs>. Último acesso a 26 de Maio de 2026.
- [6] Universidade da Beira Interior. GenJazz API – Serviço de Geração Automática de Progressões Harmónicas, 2026. Descrição fornecida no enunciado do projeto de IHC (2025/2026).
- [7] Full Stack Tools. Clerk – Authentication and User Management Toolkit, 2024. [Online] <https://www.fullstacktools.com/en/tools/authorization-authentication/tool/authorization-authentication-toolkit-clerk>. Último acesso a 26 de Maio de 2026.
- [8] Overleaf. About us. <https://www.overleaf.com/about>, 2026. Acedido em 15 de Maio de 2026.
- [9] EBAC Online. O que é GitHub: para que serve, como funciona e como utilizar, 2026. <https://ebaconline.com.br/blog/o-que-e-github>. Último acesso a 15 de Maio de 2026.

-
- [10] React Documentation Team. Your First Component, 2024. [Online] <https://pt-br.react.dev/learn/your-first-component>. Último acesso a 26 de Maio de 2026.
- [11] Document360. Guia do manual do usuário: como criar ferramentas online e práticas recomendadas, 2024. [Online] <https://document360.com/pt-br/blog/criar-manual-do-usuario/>. Último acesso a 25 de Maio de 2026.
- [12] Document360. How to Create a Troubleshooting Guide for Your Business, 2024. [Online] <https://document360.com/blog/troubleshooting-guide>. Último acesso a 27 de Maio de 2026.
- [13] Informattec Digital. Exemplos de manuais do usuário: melhores práticas e soluções, 2024. [Online] <https://informatecdigital.com/pt/Manuais-do-usu%C3%A1rio%2C-exemplos%2C-melhores-pr%C3%A1ticas-e-solu%C3%A7%C3%B5es>. Último acesso a 27 de Maio de 2026.
- [14] LinkedIn. Como escrever um manual do usuário do aplicativo, 2024. [Online] <https://pt.linkedin.com/advice/3/what-key-elements-include-application-user?lang=pt>. Último acesso a 25 de Maio de 2026.
- [15] Creative Commons. Atribuição 4.0 Internacional (CC BY 4.0) — Deed em Português, 2013. [Online] <https://creativecommons.org/licenses/by/4.0/deed.pt>. Último acesso a 27 de Maio de 2026.